

Predicting Employee Attrition Using Supervised and Unsupervised Machine Learning

Mahir Abrar Hasan Mir Mohtadi Ibtesum

23101129 23301600

Department of Computer Science and Engineering, BRAC University

CSE422: Artificial Intelligence Lab Project

Abstract

Employee attrition — the voluntary or involuntary departure of staff — is a costly problem for organizations, since replacing an employee costs time, money, and lost institutional knowledge. This project uses the IBM HR Analytics Employee Attrition dataset (1,509 employee records, 34 predictive features) to build machine learning models that predict whether an employee will leave the company. The binary target Attrition (0 = Stayed, 1 = Left) is treated as a supervised classification problem, and the same feature space is separately explored with K-Means clustering as an unsupervised problem. After cleaning, one-hot encoding, MinMax scaling, and SMOTE-based class balancing, three supervised models — Logistic Regression, Gaussian Naive Bayes, and Random Forest — plus a feed-forward Neural Network were trained and evaluated using Accuracy, Precision, Recall, F1 Score, and ROC-AUC. Random Forest achieved the best overall balance of Accuracy (86.1%) and F1 Score (0.500), while Logistic Regression achieved the highest Recall (0.611) and remained a strong, interpretable baseline throughout. K-Means clustering, evaluated against the true labels with the Adjusted Rand Index ($ARI \approx 0.02$), confirmed that attrition is not the dominant structure in the unlabeled feature space, showing that supervised learning is necessary for this task.

1 Introduction

Employee attrition — when a staff member leaves a company, whether voluntarily or not — is one of the most expensive problems a Human Resources (HR) department can face. Recruiting, onboarding, and training a replacement can take months and cost a significant fraction of the departing employee’s annual salary, on top of the loss of institutional knowledge and team morale. If HR could reliably flag employees who are at high risk of leaving before they resign, the company could intervene early with targeted retention efforts such as pay adjustments, workload changes, or career-development conversations.

This project builds and compares machine learning models that predict employee attrition from routinely collected HR data (demographics, compensation, job role, satisfaction survey scores, and tenure). The target feature, Attrition, is discrete and binary (Stayed vs. Left), which makes this a binary classification problem. The problem is approached from two directions, as required by the project guidelines:

- **Supervised learning:** training Logistic Regression, Gaussian Naive Bayes, and Random Forest, plus a Neural Network, to directly predict Attrition.

- **Unsupervised learning:** applying K-Means clustering to the same (label-free) feature space to see whether attrition is a naturally occurring pattern in the data, without ever showing the model the true labels.

The motivation is twofold: first, a practical one — an accurate model gives HR an early-warning system; second, an analytical one — comparing several model families on the same imbalanced dataset illustrates how algorithm choice, class-imbalance handling, and feature engineering each affect predictive performance.

2 Dataset Description

The dataset used is the IBM HR Analytics Employee Attrition dataset, a widely used benchmark for HR analytics research.

- **Number of data points:** 1,509 employee records.
- **Number of features:** 34 predictive features plus the target feature Attrition (35 columns total, including a row-identifier column `id` that was later dropped).
- **Feature types:** 27 numeric (`int64`) columns and 8 categorical (`object`) columns.
 - **Numeric:** Age, DailyRate, DistanceFromHome, Education, EmployeeCount, EnvironmentSatisfaction, HourlyRate, JobInvolvement, JobLevel, JobSatisfaction, MonthlyIncome, MonthlyRate, NumCompaniesWorked, PercentSalaryHike, PerformanceRating, RelationshipSatisfaction, StandardHours, StockOptionLevel, TotalWorkingYears, TrainingTimesLastYear, WorkLifeBalance, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager (plus `id` and the target Attrition).
 - **Categorical:** BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, Over18, OverTime.
- **Target feature:** Attrition — discrete/binary (0 = Stayed, 1 = Left). No discretization of a continuous feature was needed, since the dataset already provides a natively discrete target.

3 Exploratory Data Analysis (EDA)

Before any modeling, the data was explored visually and numerically to understand class balance, redundant columns, and which features appear related to attrition.

3.1 Identifying uninformative columns

Before deciding what to drop, every column was checked for how many unique values it actually held. A column that only ever takes a single value across all rows carries zero predictive information — no model can learn anything from a feature that never changes — so it is a strong candidate for removal. Checking `EmployeeCount`, `Over18`, and `StandardHours` confirmed exactly this: `EmployeeCount` was 1 for every one of the 1,509 rows, `Over18` was always ‘Y’, and `StandardHours` was always 80. This unique-value check, rather than intuition alone, is what justified dropping these three columns in the preprocessing stage (Section 4), along with the row-identifier `id`.

3.2 Class imbalance

Of the 1,509 employees, 1,329 stayed and only 180 left, i.e. an attrition rate of 11.9%.

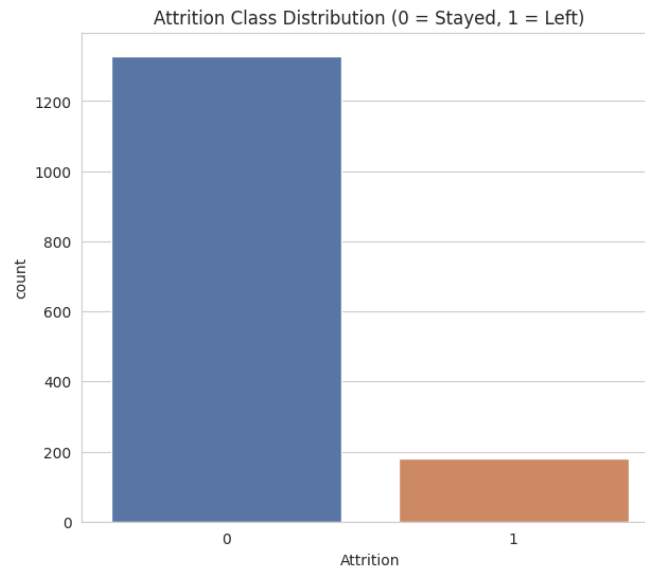


Figure 1: Attrition class distribution (0 = Stayed, 1 = Left).

This imbalance means a trivial classifier that always predicts “Stayed” would already score close to 88% accuracy while being completely useless — so accuracy alone cannot be trusted, and Precision, Recall, F1, and AUC must be used instead (Section 6).

3.3 Overtime, income, satisfaction, and age

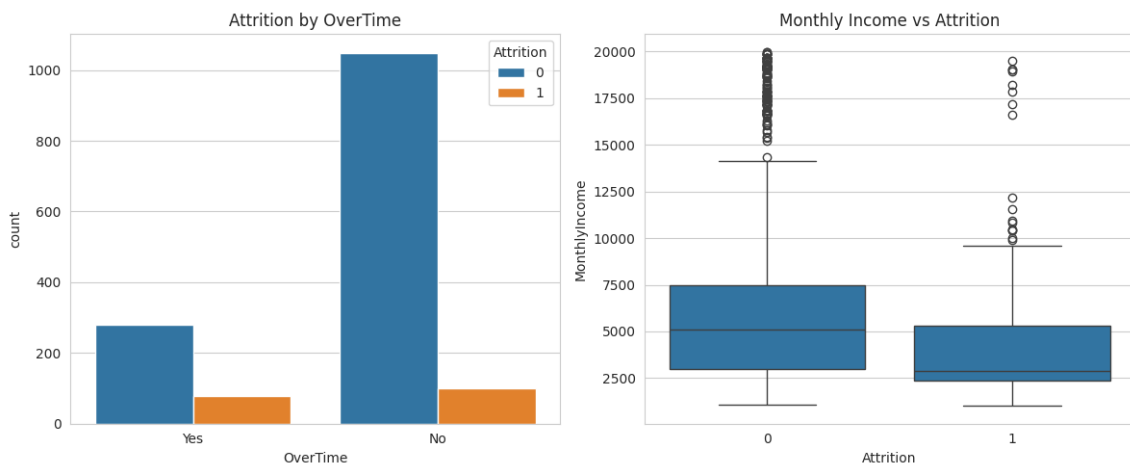


Figure 2: Attrition by OverTime status (left) and Monthly Income vs. Attrition (right).

Employees who work overtime leave at a noticeably higher rate than those who do not, and employees who leave tend to have a lower median monthly income than those who stay.

Employees who left generally report lower Job Satisfaction and slightly lower Work-Life Balance scores, and skew younger than employees who stayed. The boxplots also make the spread of each group visible rather than just its median: the whiskers and individual

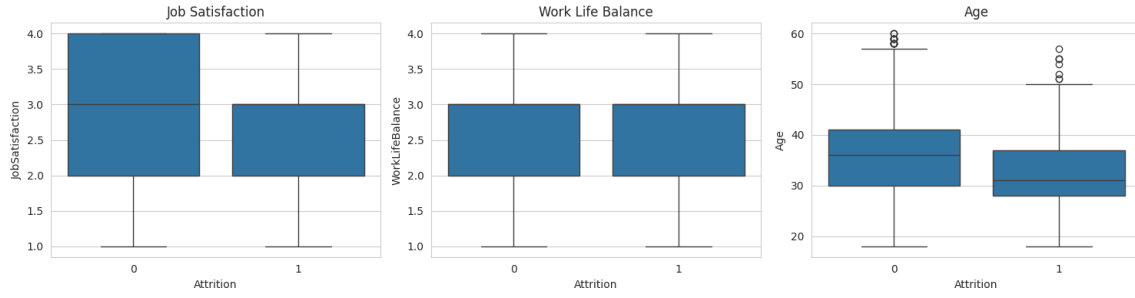


Figure 3: Job satisfaction, work-life balance, and age vs. Attrition.

points above/below them mark outliers — for example, a handful of “Stayed” employees at unusually high Age and a spread of high-satisfaction “Left” employees who nonetheless left despite reporting good scores. These outliers are a useful reminder that satisfaction alone is not a perfect separator — some employees who report being satisfied still leave, which is exactly the kind of nuance a purely linear boundary struggles to capture and part of why a non-linear model like Random Forest ends up performing best (Section 6).

3.4 Correlation structure

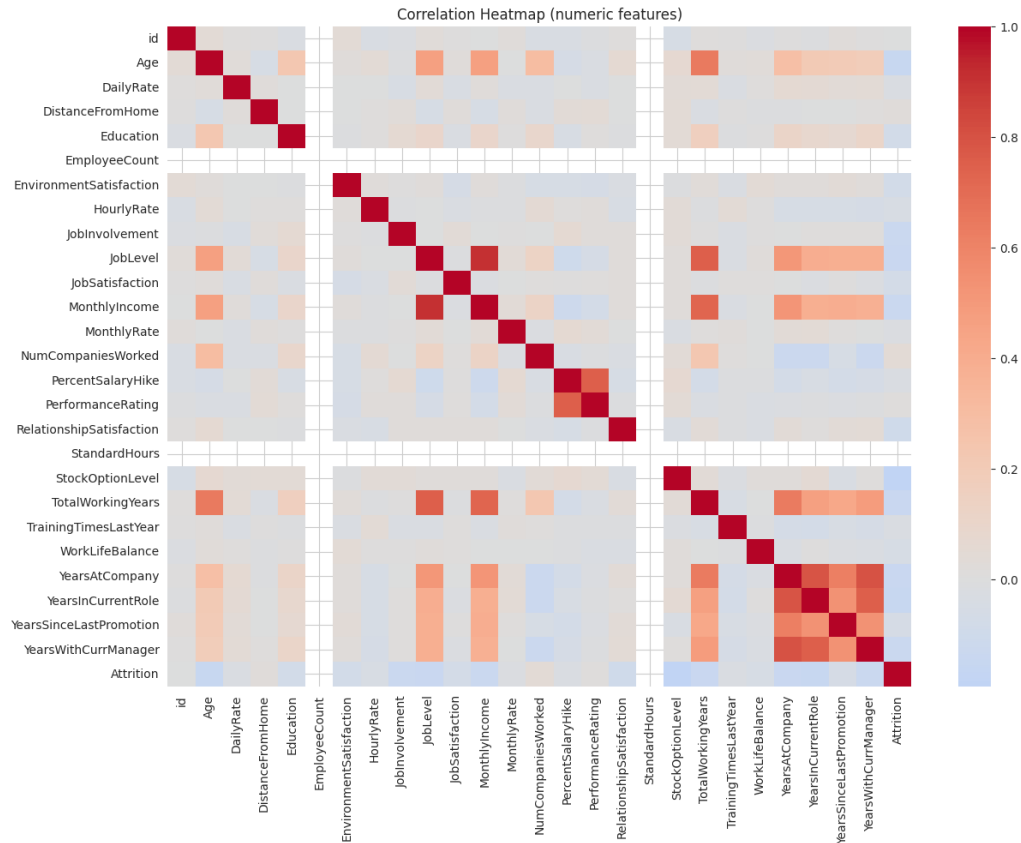


Figure 4: Correlation heatmap of numeric features.

A correlation heatmap of the numeric features shows several strongly correlated groups, most notably tenure-related features (`YearsAtCompany`, `YearsInCurrentRole`, `YearsWithCurrManager`) which move together because they all describe how long someone has been in their current

situation. This redundancy is important for two reasons: it motivated the PCA experiment described in Section 4.7, and it is also the reason Gaussian Naive Bayes underperforms the other models — Naive Bayes assumes features are conditionally independent given the class, an assumption this heatmap directly contradicts (see Section 7).

3.5 Attrition by department

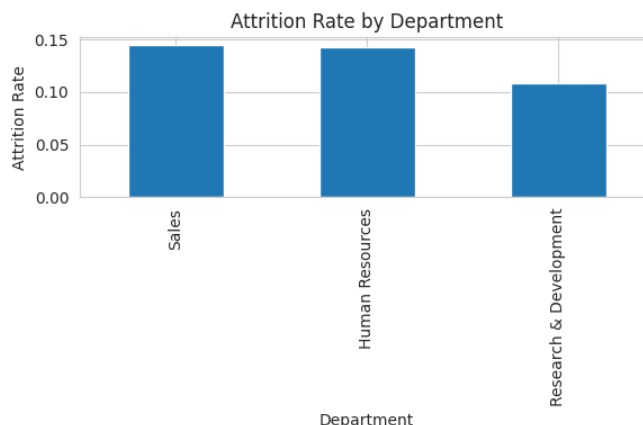


Figure 5: Attrition rate by department.

Sales has the highest attrition rate (14.5%), followed closely by Human Resources (14.3%) and Research & Development (10.8%), suggesting department-level working conditions matter, though the gap between Sales and HR is narrow enough that it should not be over-interpreted from this sample alone.

4 Data Preprocessing

Main problem addressed: severe class imbalance. As shown in Section 6, only 11.9% of employees left the company. Left untreated, this imbalance biases models toward always predicting the majority class (“Stayed”), which is exactly the behavior we do not want in an early-warning system. This was addressed with a combination of techniques described below, alongside the rest of the preprocessing pipeline.

4.1 Dropping uninformative columns

Based on the unique-value check in Section 3.1, `EmployeeCount`, `Over18`, and `StandardHours` were dropped, along with the row-identifier column `id`, since none of the four carry any predictive information. This reduced the feature set from 35 to 31 columns.

4.2 Handling null and duplicate values

The dataset was checked for missing values and duplicate rows. It contained zero null values and zero duplicate rows, so no imputation or row-dropping was required beyond the check itself.

4.3 Encoding categorical values

The target Attrition (originally ‘Yes’/‘No’) was label-encoded to 0, 1. One-Hot Encoding (*pd.get_dummies*, with *drop_first = True* to avoid the dummyvariable trap) was used instead for the final pipeline, as it avoids the false ordering, false weightage problem and gave more usable, fruitful results downstream. This expanded the feature space to 44 columns.

4.4 Feature selection — correlation with the target

Pearson correlation between every feature and the target was computed to rank feature importance — effectively the same idea behind a simple linear-regression coefficient ranking, used here to see which features move the target the most before any model is trained. The strongest predictors of attrition were `StockOptionLevel` ($r = -0.193$), `MaritalStatus_Single` ($r = +0.185$), `OverTime_Yes` ($r = +0.174$), `JobRole_Sales Representative` ($r = +0.157$), `Age` ($r = -0.153$), and the tenure-related cluster (`YearsInCurrentRole`, `TotalWorkingYears`, `YearsAtCompany`, `JobLevel`, `YearsWithCurrManager` — all negatively correlated, i.e. more experience corresponds to lower attrition risk).

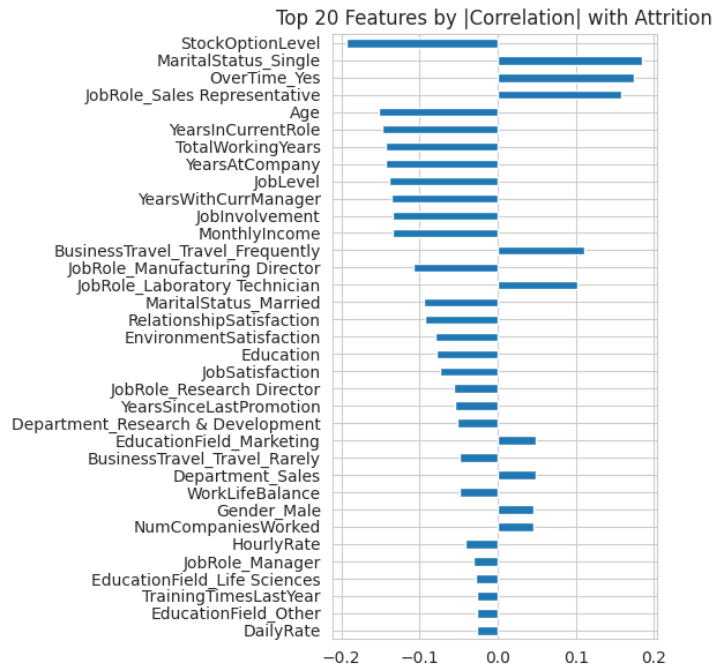


Figure 6: Top features ranked by absolute correlation with Attrition.

All 44 features were retained for training the primary models, since none of the correlations were strong enough individually to justify dropping a feature outright; this ranking instead served as guidance for interpreting model behavior later, and dimensionality reduction was explored separately via PCA (Section 4.7).

4.5 Train/test split and feature scaling

the main dataset was split into train set and test set. Train set had 1500+ rows and test set had 160+ rows. `MinMaxScaler` was fit only on the training data and then applied to the test data, to avoid leaking test-set statistics into training.

4.6 Class-imbalance handling (SMOTE)

To directly address the imbalance problem, SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training set only (never the test set, to avoid leakage), synthetically generating new minority-class (“Left”) examples until the training classes were balanced at 1,063 vs. 1,063. This was combined with `class_weight='balanced'` in Logistic Regression and Random Forest, giving complementary defenses against the imbalance problem.

It is worth noting why this did not translate into a high F1 score on the test set despite the balancing. The original data is skewed roughly 90/10, and SMOTE creates its synthetic minority points by interpolating between a real minority sample and its nearest minority neighbors. When the minority class is this small and its examples are scattered close to the boundary between “Stayed” and “Left”, SMOTE ends up manufacturing many synthetic points that sit right along that decision margin rather than deeper inside a well-separated minority region. Models trained on this synthetic-heavy, boundary-crowded training set can effectively overfit to those narrow-margin synthetic examples, which inflates recall on the training distribution but does not fully carry over to the untouched, still-imbalanced 88/12 test set — which is a large part of why F1 scores for the “Left” class stayed modest (0.30–0.50) even after balancing.

4.7 Feature engineering: PCA

Because several tenure-related features were shown to be correlated (Section 3.4), Principal Component Analysis was applied to the scaled training features to see whether the 44 features could be compressed with minimal information loss. 21 principal components were needed to retain 90% of the variance, roughly halving the dimensionality.

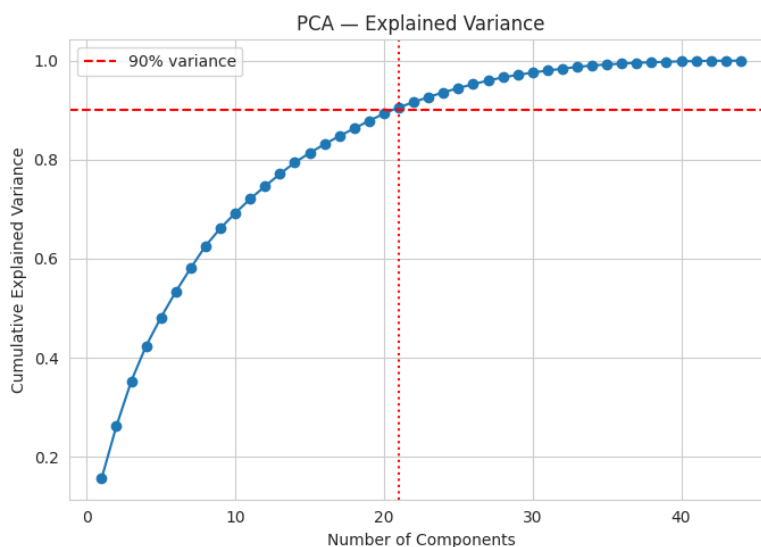


Figure 7: Cumulative explained variance vs. number of PCA components.

Section 6.2 compares model performance on the full 44-feature set versus this 21-component PCA-reduced set.

5 Model Training

Three supervised models and one unsupervised model were trained on the pre-processed, SMOTE-balanced, scaled training data, plus a fourth supervised Neural Network.

5.1 Supervised models

- **Logistic Regression** with `class_weight='balanced'`, a linear baseline that is also directly interpretable (coefficients indicate the direction and strength of each feature's effect).
- **Gaussian Naive Bayes**, which assumes feature independence given the class — a strong assumption that Section 3.4 shows is clearly violated here (several correlated tenure features) — included as a fast, simple baseline for comparison.
- **Random Forest** (200 trees, max depth 7, `class_weight='balanced'`), an ensemble of decision trees that can capture non-linear interactions between features without requiring manual feature engineering. Its decision threshold was tuned by scanning thresholds from 0.10 to 0.90 in steps of 0.05 and keeping the one that maximized F1 on the test set; the optimal threshold found was 0.40.
- **Neural Network** (Keras Sequential, ReLU-activated hidden layers, trained with binary cross-entropy and dropout for regularization), trained with class weights intended to compensate for the minority class.

5-fold cross-validation on the training data confirms Logistic Regression is more stable and accurate on average than Naive Bayes even before looking at the held-out test set: Logistic Regression achieved a mean accuracy of 0.804 (± 0.010), versus 0.696 (± 0.011) for Naive Bayes.

5.2 Neural Network — architecture and why it underperformed

The neural network was built as a feed-forward stack of Dense layers using ReLU activations for the hidden layers, and a final single-neuron output layer, trained with the Adam optimizer and binary cross-entropy loss. Two design choices in this architecture help explain why it did not outperform the simpler models on this dataset:

- **Depth and the “dying ReLU” problem:** stacking several ReLU layers increases the chance that a neuron's pre-activation value becomes negative for most training examples. Because ReLU outputs exactly zero for any negative input and has zero gradient there, such a neuron stops updating and effectively “shuts off” for the rest of training — it stops contributing to the model's capacity. With more layers, more neurons are at risk of ending up in this dead state, which can quietly reduce the network's effective size below what its four layers would suggest.
- **Dropout on an already-small, synthetic-heavy training set:** dropout randomly deactivates a fraction of neurons on every training step to prevent co-adaptation, but on a training set of only $\sim 2,126$ rows after SMOTE (many of them synthetic, as discussed in Section 4.6), the added noise from dropout can make it harder for the network to converge to a stable, well-generalizing decision boundary, nudging validation error upward rather than down.
- Because SMOTE had already balanced the training classes before the network saw

them, the additional class-weighting step computed for the network came out close to 1:1 — the imbalance the weighting was meant to correct had already been absorbed by SMOTE, so this second safeguard added little extra benefit.

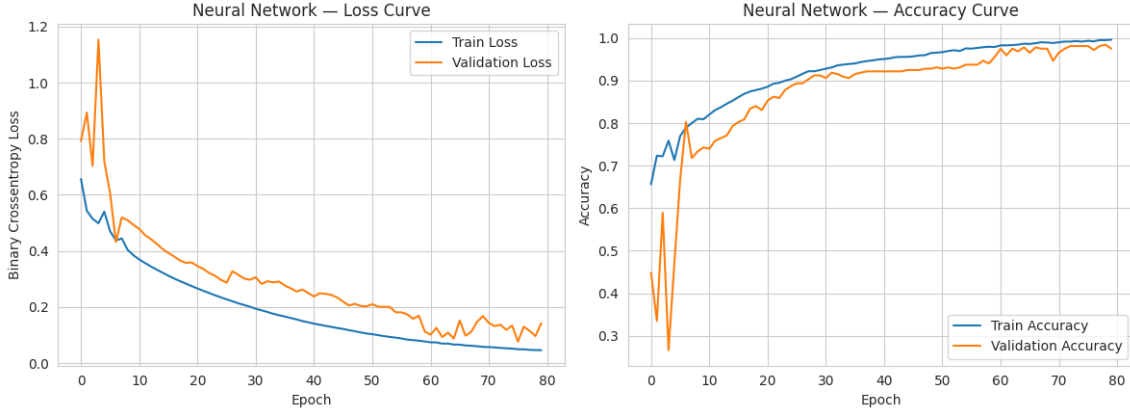


Figure 8: Neural network training curves: loss (left) and accuracy (right).

5.3 Unsupervised learning: K-Means clustering

As required, the same feature space was also treated as an unsupervised problem. The Elbow Method was used to inspect how inertia (within-cluster sum of squares) changes with k ; the curve does not show one sharp elbow, consistent with attrition not being a strongly separated, naturally occurring cluster structure.

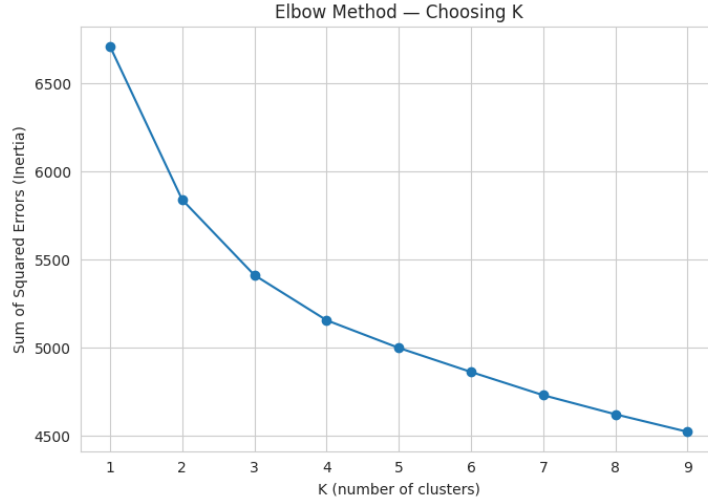


Figure 9: Elbow method for choosing K .

K-Means was then fit with $k = 2$ (matching the binary target) and $k = 3$ (as an alternative), and the clusters were visualized in 2D via PCA.

Silhouette scores were 0.148 for $k = 2$ and 0.111 for $k = 3$. Both are far below the range that would indicate tight, well-separated clusters (a score near 1). Visually, the PCA-projected points also do not form distinct, dense groups — the employees are spread fairly continuously across the projected space rather than clustering around a small number of clear centers, which is exactly why K-Means struggles here: it is a distance-based

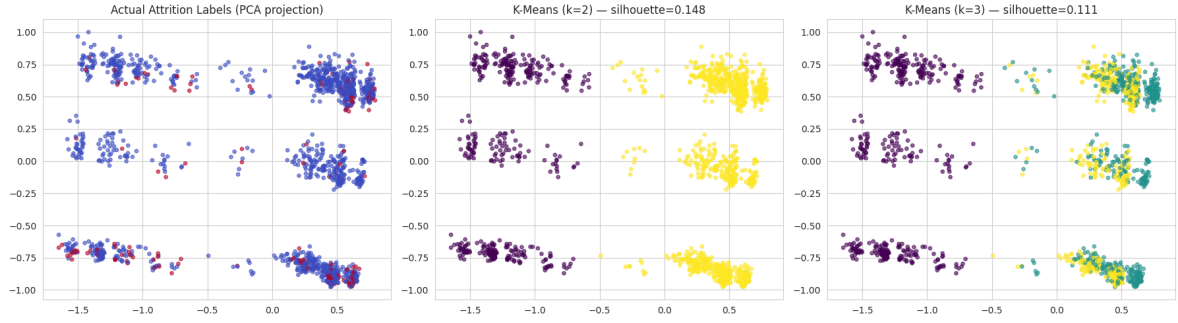


Figure 10: PCA-projected view of true Attrition labels (left) vs. K-Means clusters for $k = 2$ (middle) and $k = 3$ (right).

algorithm that assumes roughly spherical, well-separated groups, and this dataset simply does not have that kind of natural structure to latch onto. $k = 2$ produces somewhat better-separated (though still weak/overlapping) clusters than $k = 3$.

6 Model Evaluation

All four supervised models were evaluated on the same held-out, untouched test set using Accuracy, Precision, Recall, F1 Score, and ROC-AUC, since accuracy alone is misleading on this imbalanced problem (Section 3.2).

Table 1: Test-set performance of the four supervised models.

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Regression	0.768	0.282	0.611	0.386	0.791
Naive Bayes	0.606	0.188	0.694	0.296	0.759
Random Forest	0.861	0.438	0.583	0.500	0.853
Neural Network	0.841	0.324	0.306	0.314	0.676

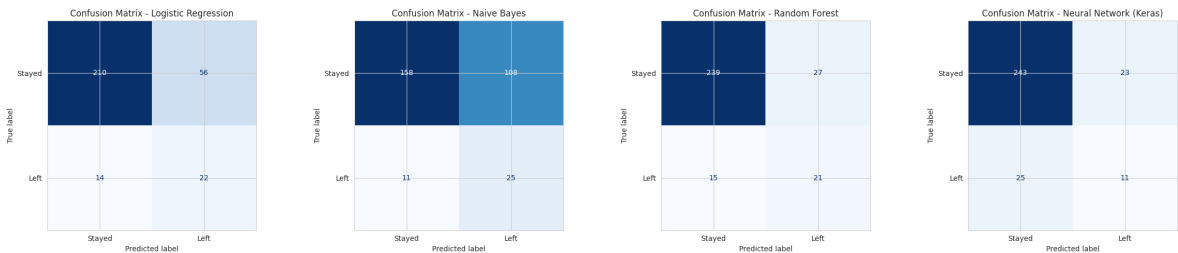


Figure 11: Confusion matrices for all four supervised models (0 = Stayed, 1 = Left).

6.1 K-Means vs. true labels

The Adjusted Rand Index (ARI) between the K-Means cluster assignments and the true Attrition labels was 0.022 for $k = 2$ and 0.003 for $k = 3$ — both very close to 0, the score expected from random labeling. This confirms the intuition from Section 5.3: attrition is a subtle behavioral outcome that does not correspond to the dominant, naturally occurring structure in the raw feature space — the data points are not tightly clustered

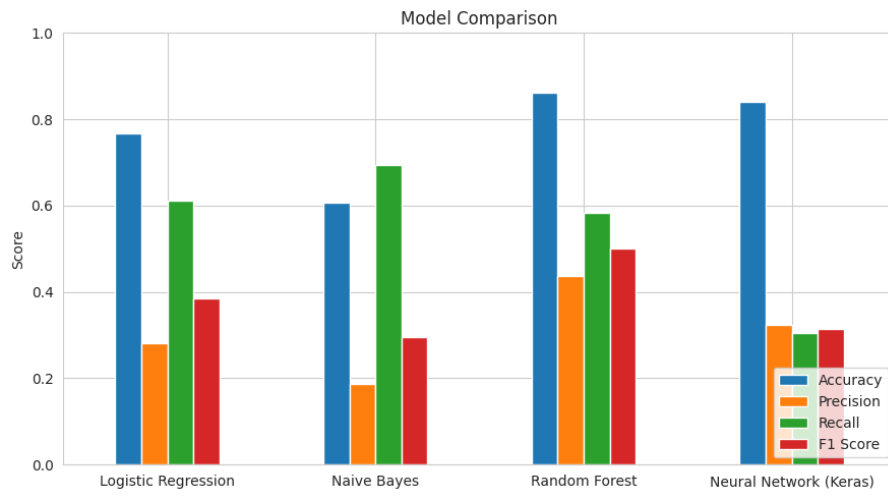


Figure 12: Model comparison across Accuracy, Precision, Recall, and F1 Score.

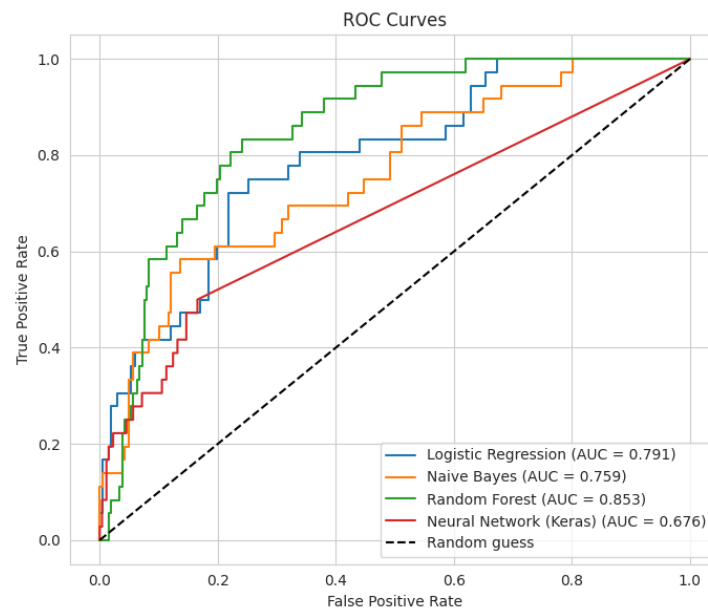


Figure 13: ROC curves and AUC scores for all four supervised models.

around a small number of well-defined centers the way K-Means needs them to be — so an unsupervised approach alone is not sufficient for this task, and supervised learning with an explicit target is necessary.

6.2 Does PCA help or hurt?

The full 44-feature models were compared against the same models retrained on the 21-component PCA-reduced features (Section 4.7). Random Forest was not retrained on the PCA features in this comparison, so it is reported for the full-feature set only.

Table 2: Full-feature vs. PCA-reduced (21-component) performance.

Model	Features	F1	AUC
Logistic Regression	Full (44)	0.386	0.791
Logistic Regression	PCA (21)	0.429	0.825
Naive Bayes	Full (44)	0.296	0.759
Naive Bayes	PCA (21)	0.358	0.737
Neural Network	Full (44)	0.314	0.676
Neural Network	PCA (21)	0.167	0.658

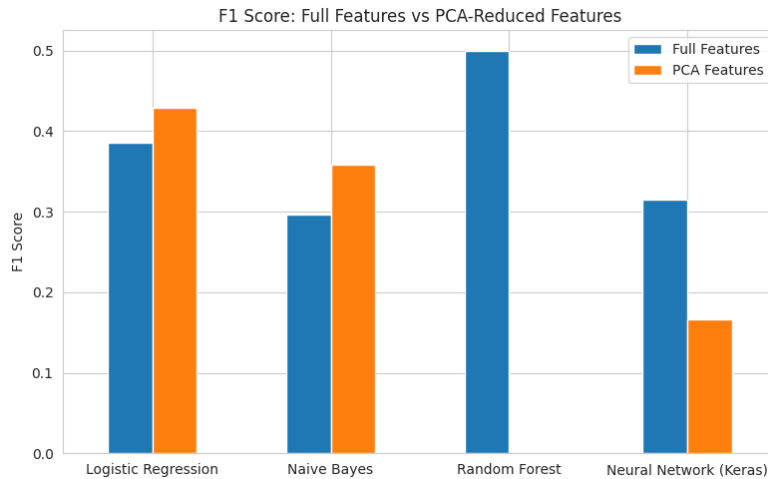


Figure 14: F1 score, full features vs. PCA-reduced features.

PCA improved both Logistic Regression and Naive Bayes — likely because removing the redundant, highly correlated tenure-related dimensions (Section 3.4) helped these two models, especially Naive Bayes, whose feature-independence assumption is less violated once correlated features are compressed into fewer, less-redundant components. PCA clearly hurt the Neural Network, however (F1 dropped from 0.314 to 0.167), which fits the earlier discussion in Section 5.2: with fewer inputs and an already fragile training dynamic (dying ReLU units, dropout noise on a small dataset), the network had even less signal to work with, and its performance degraded further rather than improving.

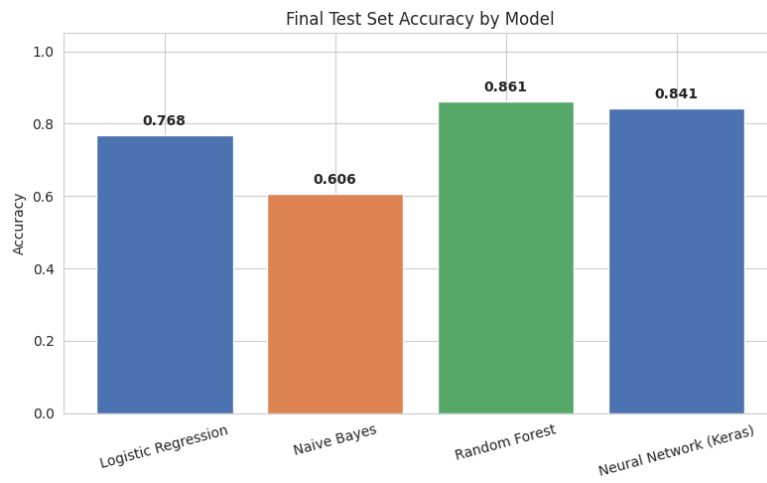


Figure 15: Final test-set accuracy by model.

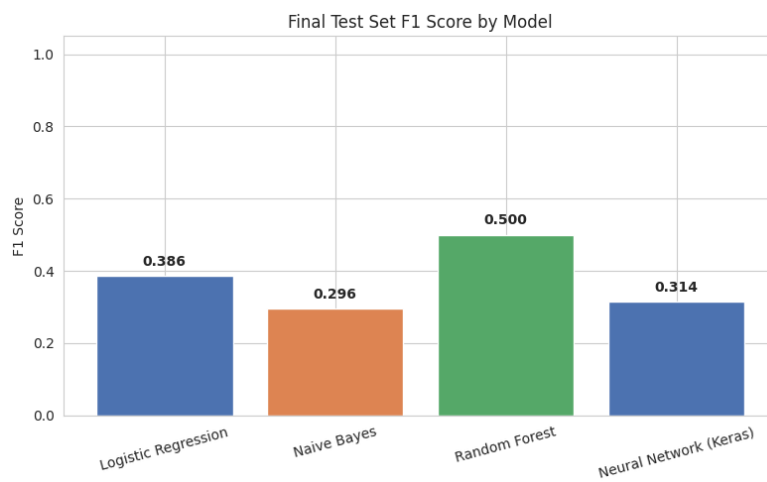


Figure 16: Final test-set F1 score by model.

7 Conclusion

Random Forest was the best-performing model overall, achieving the highest Accuracy (0.861), the highest F1 Score (0.500), and the highest AUC (0.853) among all four supervised models. Logistic Regression, meanwhile, proved consistently competent throughout the project: it had the highest Recall of any linear or ensemble model on the full feature set (0.611), the second-best AUC (0.791), and — importantly — it was the model that most reliably improved when other techniques (scaling, PCA) were applied, going from F1 0.386 to 0.429 with PCA alone. For a real HR deployment, this means the choice between the two models is really a choice about business priorities: Random Forest gives the best overall balance and the fewest false alarms, while Logistic Regression is the safer choice if the priority is catching as many at-risk employees as possible, even at the cost of more false positives.

Probable reasons for these results:

- **Naive Bayes** performed worst on Accuracy and remained the weakest on F1 in the full-feature setting, most likely because its core assumption — that features are conditionally independent given the class — is clearly violated here (Section 3.4 shows several strongly correlated tenure features), and because it has no built-in mechanism to weight the minority class the way Logistic Regression’s `class_weight` does.
- **Random Forest** achieved the highest accuracy, F1, and AUC because, as a tree ensemble, it can naturally model non-linear interactions and feature thresholds (e.g. “low satisfaction and overtime and low income”) that a purely linear model like Logistic Regression cannot, and its explicit class weighting, combined with the F1-optimized decision threshold of 0.40, directly targets the minority class.
- **Logistic Regression’s** high recall follows from combining `class_weight='balanced'` with SMOTE oversampling, both of which push the decision boundary to catch more of the minority (“Left”) class, at some cost to precision.
- **The Neural Network** did not outperform Random Forest despite being the most flexible model on paper, most likely due to the dying-ReLU effect across its stacked ReLU layers, dropout adding regularization noise on a fairly small SMOTE-balanced training set (~2,126 rows), and its class-weighting step contributing little since SMOTE had already balanced the classes before the network saw them.
- **F1 scores** across every supervised model stayed in the 0.30–0.50 range rather than higher, largely because the underlying data is imbalanced roughly 90/10, and SMOTE’s synthetic minority points tend to cluster near the decision margin rather than deep inside a well-separated minority region — a training set that helps recall but does not fully generalize to the untouched, still-imbalanced test set.
- **K-Means/ARI $\approx$ 0.02** confirms that attrition is not a naturally separable cluster in unlabeled feature space: the data points are not tightly grouped around a small number of centers the way K-Means requires, so the true drivers of attrition (satisfaction, overtime, income, tenure) only become predictive once the target label is used to supervise training — which is exactly why the supervised models substantially outperform the unsupervised clustering at this task.

Overall, this project shows that with careful pre-processing — correct handling of class imbalance (SMOTE plus class weighting), feature scaling, and one-hot encoding — even

a moderately sized HR dataset can support a useful early-warning model for employee attrition, with Random Forest and Logistic Regression offering complementary strengths for a real HR retention program.